

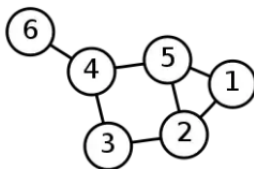
Algorithmique

Rakotoarimalala Tsinjo Tony

Introduction Graphe

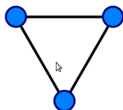
Définition

- **Wikipedia:** un graphe est une structure composée d'objets dans laquelle certaines paires d'objets sont en relation. Les objets correspondent à des abstractions mathématiques et sont appelés **sommets** (ou nœuds ou points), et les relations entre sommets sont des **arêtes** (ou liens ou lignes).

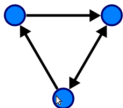


Définitions

- Un graphe simple non orienté est un couple $G = (E, V)$ avec
 - V un ensemble de sommets (aussi appelés nœuds ou points)
 - $E \subset \{(x, y) | (x, y) \in V^2, x \neq y\}$ un ensemble d'arêtes



- Un graphe simple orienté est un couple $G = (E, V)$ avec
 - V un ensemble de sommets (aussi appelés nœuds ou points)
 - $E \subset \{(x, y) | (x, y) \in V^2, x \neq y\}$ un ensemble de flèches ou arcs.

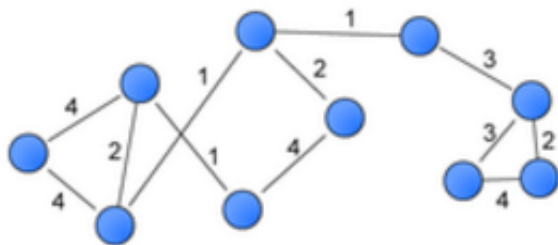


- Une boucle est une arête partant d'un sommet vers lui-même
- Si deux sommets sont liés par une arête alors on dit que les deux sommets sont **adjacents** ou encore **voisins**, on note $x \sim y$
- Le degré d'un sommet est le nombre de voisins de ce sommet

- Une boucle est une arête partant d'un sommet vers lui-même
- Dans la flèche (x, y) orientée de x vers y , x est appelé la queue de la flèche et y la tête de la flèche. La flèche (y, x) est appelée la flèche inverse de (x, y) .

Graphe pondéré

- Un graphe pondéré ou un réseau est un graphe où chaque arête porte un nombre (son poids)
- Ces poids peuvent représenter par exemple des coûts, des longueurs ou des capacités, en fonction du problème traité.



Exemple d'objets modélisables avec un graphe

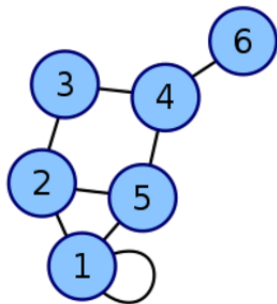
D'après vous ?

Matrice d'adjacence

- Supposons que $G = (V, E)$ est un graphe simple, où $|V| = n$.
- Supposons aussi que les sommets de G sont numérotés arbitrairement $v_1, v_2, \dots, v_n$.
- La matrice d'adjacence A de G se rapportant à cet ensemble de sommets est la matrice $n \times n$ booléenne A avec

$$a_{ij} = \begin{cases} 1 & \text{si } (v_i, v_j) \in E \\ 0 & \text{sinon} \end{cases}$$

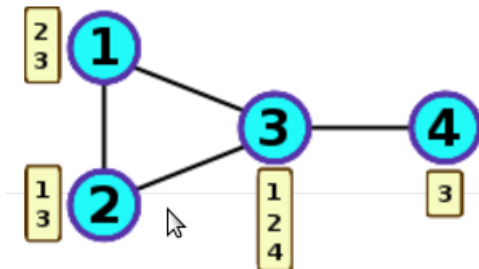
Matrice d'adjacence



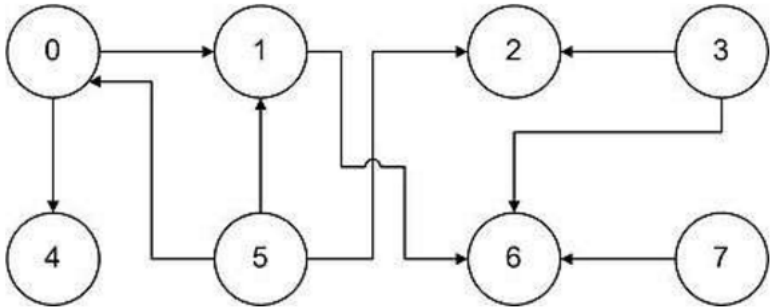
$$\begin{pmatrix} 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

Liste d'adjacence

- La liste d'adjacence d'un graphe non orienté, est la liste des voisins de chaque sommet



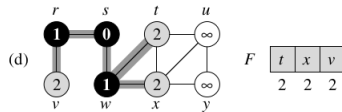
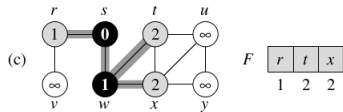
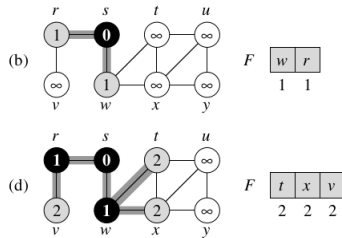
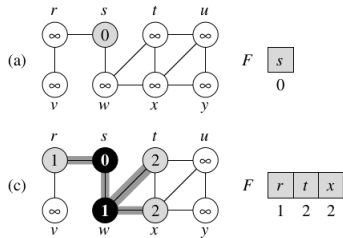
Représenter le graphe suivant avec une matrice et une liste

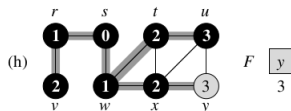
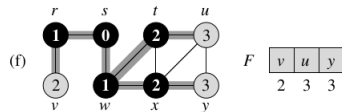
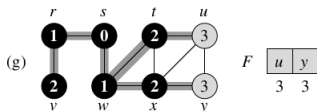
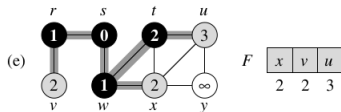


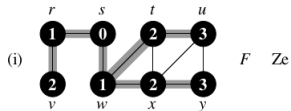
Algorithm Parcours en largeur (BFS)

```
procedure BFS( $G, s$ )      ▷  $G$ : graphe,  $s$ : sommet de depart
     $visited[s] \leftarrow \mathbf{true}$ 
     $Q \leftarrow$  file vide
    Enfiler( $Q, s$ )
    while  $Q$  n'est pas vide do
         $v \leftarrow$  Défiler( $Q$ )
        for chaque voisin  $u$  de  $v$  dans  $G$  do
            if  $u$  n'est pas visite then
                Marquer  $u$  comme visite
                Enfiler( $Q, u$ )
            end if
        end for
    end while
end procedure
```

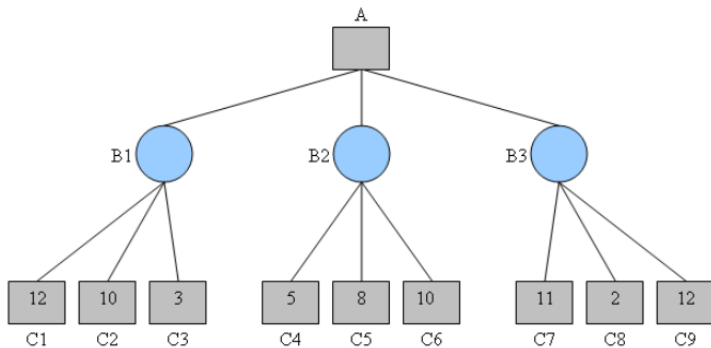
BFS



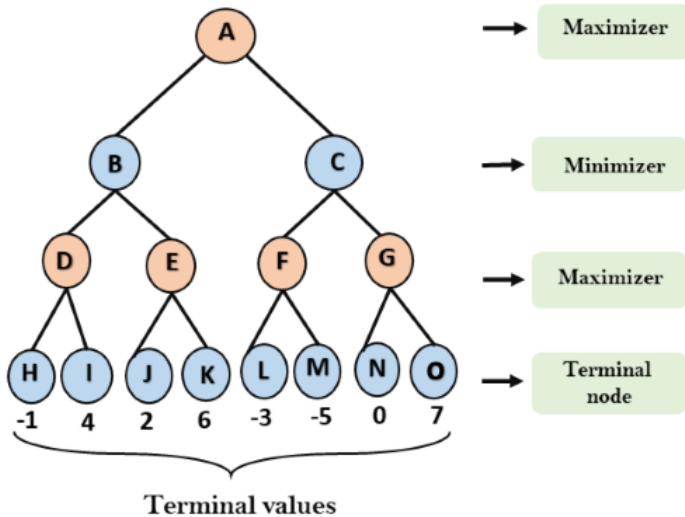




Arbre de jeu simple



Arbre de jeu plus compliqué



Algorithm Minimax

```
1: function MINIMAX(état, profondeur, estMax)
2:   if profondeur = 0 ou état est terminal then
3:     return la valeur heuristique de état
4:   end if
5:   if estMax then
6:      $v \leftarrow -\infty$ 
7:     for chaque mouvement valide  $m$  depuis état do
8:        $v \leftarrow \max(v, \text{Minimax}(m, \text{profondeur} - 1, \text{Faux}))$ 
9:     end for
10:    return  $v$ 
11:  else
12:     $v \leftarrow +\infty$ 
13:    for chaque mouvement valide  $m$  depuis état do
14:       $v \leftarrow \min(v, \text{Minimax}(m, \text{profondeur} - 1, \text{Vrai}))$ 
15:    end for
16:    return  $v$ 
17:  end if
18: end function
```
